Lab Assignment #4b

Math 437 - Modern Data Analysis - Jason Soto

Introduction

In a 2020 article, Murray et al. attempted to understand the relationship between a professor's research output and their perceived teaching effectiveness.

To quantify perceived teaching effectiveness, they scraped Rate My Professor and calculated an "overall effectiveness" score (based on the data dictionary, this appears to just be the average star rating, on a scale from 1-5, from reviews posted between Spring 2010 and Fall 2017 semesters).

An anonymized, processed dataset and accompanying data dictionary is publicly available; I've removed most of the research-related variables to shrink the size of the dataset and shared it with you on Canvas as *RateMyProfessor*.

```
here::i_am("Labs/Lab04b.qmd")

# load packages
library(ISLR2)
library(car)
library(ggplot2)

# load data
RateMyProfessor <- readr::read_csv(here::here("Data/RateMyProfessor.csv"))
```

We will continue to build increasingly complex models to understand what goes into a professor's RateMyProfessor rating.

This lab assignment is worth a total of **12.5 points**.

Problem 4: Interaction Terms (5.5 pts total)

Part a (Code: 0.5 pts)

Run the single line of code in ISLR Lab 3.6.4.

Solution

```r
summary(lm(medv ~ lstat * age, data = Boston))
```

```
Call:
lm(formula = medv ~ lstat * age, data = Boston)

Residuals:
    Min      1Q  Median      3Q     Max 
-15.806  -4.045  -1.333   2.085  27.552 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept) 36.0885359  1.4698355  24.553  < 2e-16 ***
lstat       -1.3921168  0.1674555  -8.313 8.78e-16 ***
age         -0.0007209  0.0198792  -0.036   0.9711    
lstat:age    0.0041560  0.0018518   2.244   0.0252 *  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.149 on 502 degrees of freedom
Multiple R-squared:  0.5557,	Adjusted R-squared:  0.5531 
F-statistic: 209.3 on 3 and 502 DF,  p-value: < 2.2e-16
```

Part b (Explanation: 1 pt)

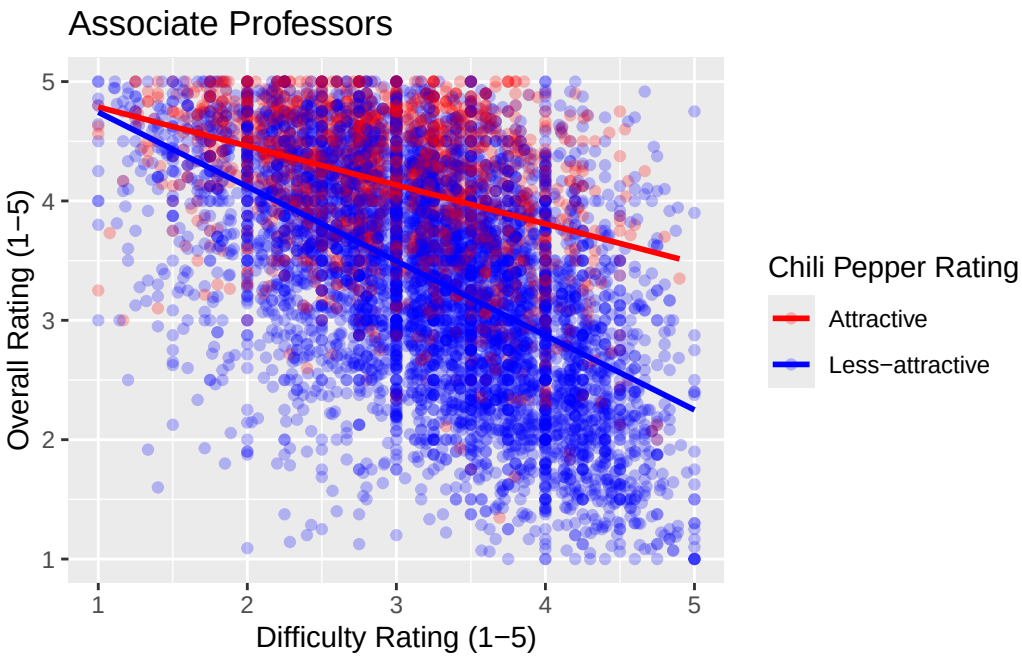
The code below creates a new dataset, `associates`, by subsetting the `RateMyProfessor` dataset to include only the Associate Professors.

```r
# we will learn a much more elegant way to do this!
associates <- RateMyProfessor[
  which(RateMyProfessor$rank == "Associate Professor"),
  # all columns
]
```

This code creates a graph of overall rating vs. difficulty rating for those associate professors, with "hot" professors shown in red and "cold" professors shown in blue.

```
# Step 1: Set up the plot
ggplot(
  data = associates,
  mapping = aes(
    x = difficulty,
    y = overall,
    color = factor(hotness) |> relevel(ref = "hot")
  )
) +
# Step 2a: Add the scatterplot
  geom_point(alpha = 0.25) +
# Step 2b: add the least-squares regression lines
  geom_smooth(method = "lm", se = FALSE) + # se = FALSE means no confidence bands
# Step 3: customize the plot and legend labels
  labs( # make labels nice
    title = "Associate Professors",
    x = "Difficulty Rating (1-5)",
    y = "Overall Rating (1-5)"
  ) +
  scale_color_manual(  # make legend nice
    name = "Chili Pepper Rating",
    labels = c(hot = "Attractive", cold = "Less-attractive"),
    values = c(hot = "red", cold = "blue")
  )
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Based on this graph, how does the difficulty of the professor modify the relationship between attractiveness and overall rating?

Solution

When difficulty increases, overall rating does as well, but the relationship we see between attractiveness and overall rating is that less-attractive professors are penalized more than the attractive.

Part c (Code: 0.5 pts; Computation and Explanation: 2 pts)

Using the `associates` dataset, fit a linear model predicting overall rating from the difficulty rating (`difficulty`), chili pepper rating (`hotness`), and an interaction term between `difficulty` and `hotness`.

Using the model output, write out the least-squares regression equation for the red line and the blue line on the plot from Part a. Explain how you obtained each equation and which equation corresponds to which line.

Solution

```r
lm_group_interaction <- lm(
  overall ~ difficulty * hotness,
  data = associates
)

summary(lm_group_interaction)
```

```
Call:
lm(formula = overall ~ difficulty * hotness, data = associates)

Residuals:
     Min       1Q   Median       3Q      Max 
-3.02928 -0.49182  0.04825  0.50502  2.50044 

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)    
(Intercept)            5.36728    0.03995 134.358  < 2e-16 ***
difficulty            -0.62354    0.01199 -51.985  < 2e-16 ***
hotnesshot            -0.25415    0.08325  -3.053  0.00227 ** 
difficulty:hotnesshot  0.29720    0.02692  11.040  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7288 on 8058 degrees of freedom
Multiple R-squared:  0.3655,	Adjusted R-squared:  0.3653 
F-statistic:  1547 on 3 and 8058 DF,  p-value: < 2.2e-16
```

Part d (Explanation: 1.5 pts)

The slope corresponding to `hotnesshot` in the model from Part c is approximately -0.254, implying that "hot" professors are predicted to have lower overall ratings than "cold" professors. But this contradicts the graph in Part a, in which "hot" professors consistently have higher ratings than "cold" professors.

Explain what's missing from the interpretation of the slope of -0.254, and how fixing the interpretation resolves the apparently counter-intuitive finding.

Solution

yhat = 5.3672 + (-.6235)(difficulty) + (-.2541)(hotness = "hot") + .2972(difficulty)(hotness = "hot") yhat = 5.3672 + (-.6235)(difficulty) (unattractive) yhat = 5.1131 + (-.3263)(difficulty) (attractive)

When group changes from unattractive to attractive, and difficulty is held constant at 0, predicted y by .2541. This is what is misinterpreted, we have a prediction for the y intercept which we do not care about since difficulties range from 1 to 5. After working out our yhats, we find that attractive professors have a more forgiving negative effect from difficulty.

Thus, what's missing is that the coefficient –0.254 is not the overall difference between "hot" and "cold" professors. It is the difference when difficulty = 0, which is not possible.

Problem 5: Polynomial Regression (2.5 pts total)

Part a (Code: 0.5 pts)

Run the code in ISLR Lab 3.6.5. Put each chunk from the textbook in its own chunk.

Solution

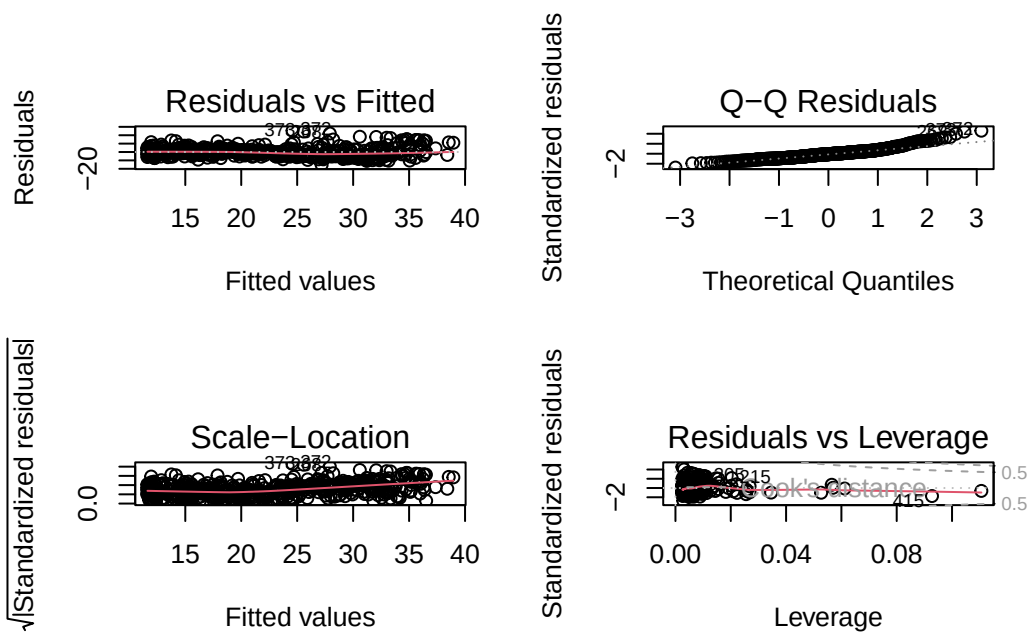
```r
lm.fit2 <- lm(medv ~ lstat + I(lstat^2), data = Boston)
summary(lm.fit2)
```

```
Call:
lm(formula = medv ~ lstat + I(lstat^2), data = Boston)

Residuals:
     Min       1Q   Median       3Q      Max 
-15.2834  -3.8313  -0.5295   2.3095  25.4148 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 42.862007   0.872084   49.15   <2e-16 ***
lstat       -2.332821   0.123803  -18.84   <2e-16 ***
I(lstat^2)   0.043547   0.003745   11.63   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.524 on 503 degrees of freedom
Multiple R-squared:  0.6407,	Adjusted R-squared:  0.6393 
F-statistic: 448.5 on 2 and 503 DF,  p-value: < 2.2e-16
```

```r
lm.fit <- lm(medv ~ lstat, data = Boston)
anova(lm.fit, lm.fit2)
```

```
Analysis of Variance Table

Model 1: medv ~ lstat
Model 2: medv ~ lstat + I(lstat^2)
  Res.Df   RSS Df Sum of Sq     F    Pr(>F)
1    504 19472
2    503 15347  1    4125.1 135.2 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
par(mfrow = c(2, 2))
plot(lm.fit2)
```




```r
lm.fit5 <- lm(medv ~ poly(lstat, 5), data = Boston)
summary(lm.fit5)
```

```
Call:
```

```
lm(formula = medv ~ poly(lstat, 5), data = Boston)

Residuals:
     Min       1Q   Median       3Q      Max 
-13.5433  -3.1039  -0.7052   2.0844  27.1153 

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)    
(Intercept)       22.5328     0.2318  97.197  < 2e-16 ***
poly(lstat, 5)1 -152.4595     5.2148 -29.236  < 2e-16 ***
poly(lstat, 5)2   64.2272     5.2148  12.316  < 2e-16 ***
poly(lstat, 5)3  -27.0511     5.2148  -5.187 3.10e-07 ***
poly(lstat, 5)4   25.4517     5.2148   4.881 1.42e-06 ***
poly(lstat, 5)5  -19.2524     5.2148  -3.692 0.000247 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.215 on 500 degrees of freedom
Multiple R-squared:  0.6817,	Adjusted R-squared:  0.6785 
F-statistic: 214.2 on 5 and 500 DF,  p-value: < 2.2e-16
```

```r
summary(lm(medv ~ log(rm), data = Boston))
```

```
Call:
lm(formula = medv ~ log(rm), data = Boston)

Residuals:
    Min      1Q  Median      3Q     Max 
-19.487  -2.875  -0.104   2.837  39.816 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  -76.488      5.028  -15.21   <2e-16 ***
log(rm)       54.055      2.739   19.73   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.915 on 504 degrees of freedom
Multiple R-squared:  0.4358,	Adjusted R-squared:  0.4347 
F-statistic: 389.3 on 1 and 504 DF,  p-value: < 2.2e-16
```

Part b (Code: 1 pt)

Run the code in ISLR Lab 7.8.1. Stop when you get to the model predicting whether someone earns more than $250,000 per year.

Do not attach the `Wage` dataset. If a line of code involving `age` or `wage` does not run, use `Wage$age` or `Wage$wage` instead (as appropriate).

In this lab, "both subplots" refers to the graph showing the linear regression model `fit` and the graph showing the logistic regression model `fit`. Since we are not fitting the logistic regression model in this lab, you can ignore the line setting up the margins of the plot with `par(mfrow = c(1,2), ...)`.

The Chapter 7 lab does a poor job of bookkeeping. You should use more informative names than just `fit` so that you know which model you are `fit`ting. Otherwise, it will be easy to run into problems where you think you're predicting with one model and you're actually predicting with a different one.

Solution

```r
fit <- lm(wage ~ poly(age, 4),
          data = Wage)
coef(summary(fit))
```

```
                Estimate Std. Error    t value     Pr(>|t|)
(Intercept)    111.70361  0.7287409 153.283015 0.000000e+00
poly(age, 4)1  447.06785 39.9147851  11.200558 1.484604e-28
poly(age, 4)2 -478.31581 39.9147851 -11.983424 2.355831e-32
poly(age, 4)3  125.52169 39.9147851   3.144742 1.678622e-03
poly(age, 4)4  -77.91118 39.9147851  -1.951938 5.103865e-02
```

```r
fit2 <- lm(wage ~ poly(age, 4, raw = T),
           data = Wage)
coef(summary(fit2))
```

```
                           Estimate   Std. Error   t value     Pr(>|t|)
(Intercept)            -1.841542e+02 6.004038e+01 -3.067172 0.0021802539
poly(age, 4, raw = T)1  2.124552e+01 5.886748e+00  3.609042 0.0003123618
poly(age, 4, raw = T)2 -5.638593e-01 2.061083e-01 -2.735743 0.0062606446
poly(age, 4, raw = T)3  6.810688e-03 3.065931e-03  2.221409 0.0263977518
poly(age, 4, raw = T)4 -3.203830e-05 1.641359e-05 -1.951938 0.0510386498
```

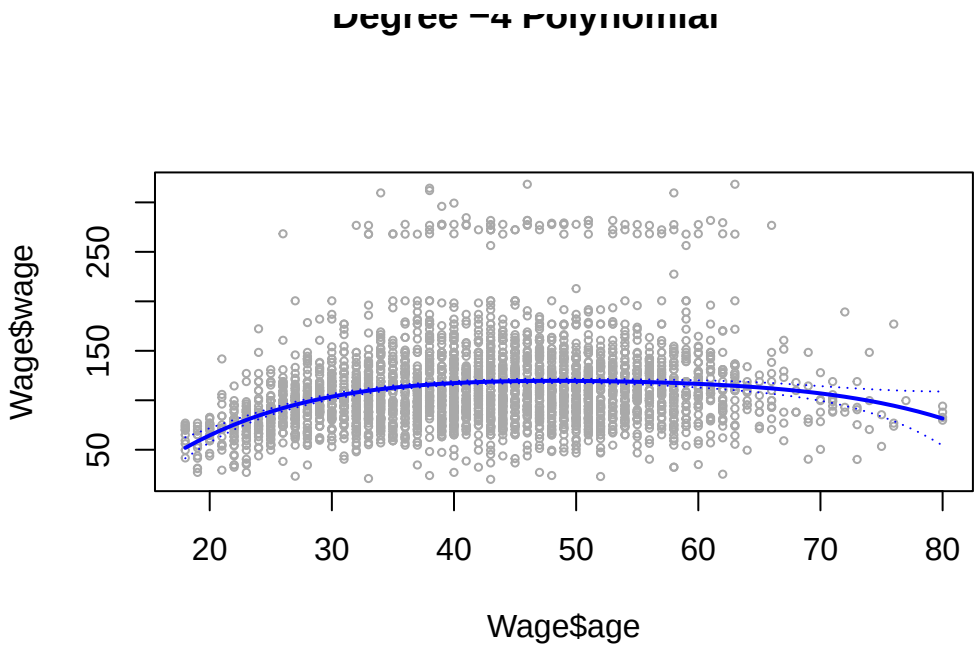
```r
fit2a <- lm(wage ~ age + I(age^2) + I(age^3) + I(age^4),
            data = Wage)
coef(fit2a)
```

```
  (Intercept)           age      I(age^2)      I(age^3)      I(age^4)
-1.841542e+02  2.124552e+01 -5.638593e-01  6.810688e-03 -3.203830e-05
```

```r
fit2b <- lm(wage ~ cbind(age, age^2, age^3, age^4),
            data = Wage)
```

```r
agelims <- range(Wage$age)

age.grid <- seq(from = agelims[1],
                to = agelims[2])

preds <- predict(fit,
                 newdata = list(age = age.grid),
                 se = TRUE)
se.bands <- cbind(preds$fit + 2 * preds$se.fit, preds$fit - 2 * preds$se.fit)
```

```r
plot(Wage$age, Wage$wage, xlim = agelims, cex = .5, col = "darkgrey")
title("Degree -4 Polynomial", outer = T)
lines(age.grid, preds$fit, lwd = 2, col = "blue")
matlines(age.grid, se.bands, lwd = 1, col = "blue", lty = 3)
```



Part c (Code: 1 pt)

Produce a figure similar to the left panel of Figure 7.1, with the following differences:

- You should predict `overall` from `difficulty` using the `associates` dataset.
- You should use a quadratic (degree-2) polynomial instead of a quartic (degree-4).
- Instead of plotting approximate 95% confidence bands using the 2 SE method, plot exact 95% confidence bands by replacing `se = TRUE` with `interval = "confidence"` and figuring out how to subset the resulting data frame to create `se.bands`.

Notice that the predictions are now returned as a matrix instead of a list, so you will need to do a bit of investigation to figure out how to subset the matrix correctly to produce the lines on the plot.

Solution

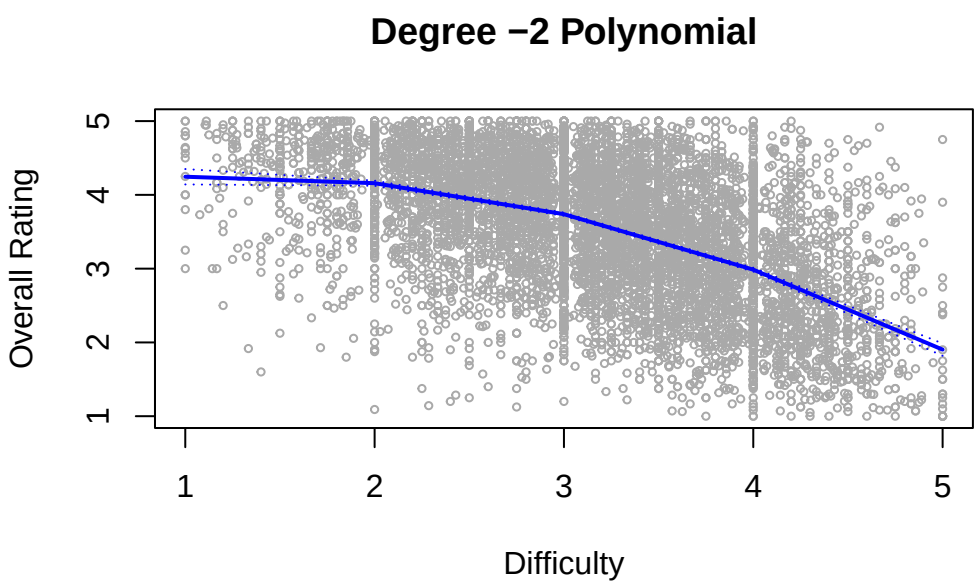
```
fit_oa <- lm(overall ~ poly(difficulty, 2),
             data = associates)
coef(summary(fit_oa))
```

```
                    Estimate Std. Error   t value     Pr(>|t|)
```

```
(Intercept)            3.535444 0.00859746 411.21960 0.000000e+00
poly(difficulty, 2)1 -42.714202 0.77195424 -55.33256 0.000000e+00
poly(difficulty, 2)2 -10.880064 0.77195424 -14.09418 1.391988e-44
```

```r
difflims <- range(associates$difficulty)

diff.grid <- seq(from = difflims[1],
                 to = difflims[2])

preds_oa <- predict(fit_oa,
                    newdata = list(difficulty = diff.grid),
                    se = TRUE)
se.bands_oa <- cbind(preds_oa$fit + 2 * preds_oa$se.fit, preds_oa$fit - 2 * preds_oa$se.fit)
```

```r
plot(associates$difficulty, associates$overall, xlim = difflims, cex = .5, xlab = "Difficulty
title("Degree -2 Polynomial")
lines(diff.grid, preds_oa$fit, lwd = 2, col = "blue")
matlines(diff.grid, se.bands_oa, lwd = 1, col = "blue", lty = 3)
```




```r
#difflims <- range(associates$difficulty)

#diff.grid <- seq(from = difflims[1],
```

```
                #to = difflims[2])

#preds_oa <- predict(fit_oa,
                 #newdata = list(difficulty = diff.grid),
                 #se = FALSE,
                 #interval = "confidence")
```

```
#plot(associates$difficulty, associates$overall, xlim = difflims, cex = .5, col = "darkgrey")
#title("Degree -2 Polynomial", outer = T)
#lines(diff.grid, preds_oa[,1], lwd = 2, col = "blue")
#matlines(diff.grid, preds_oa[, 2:3], lwd = 1, col = "blue", lty = 3)
```

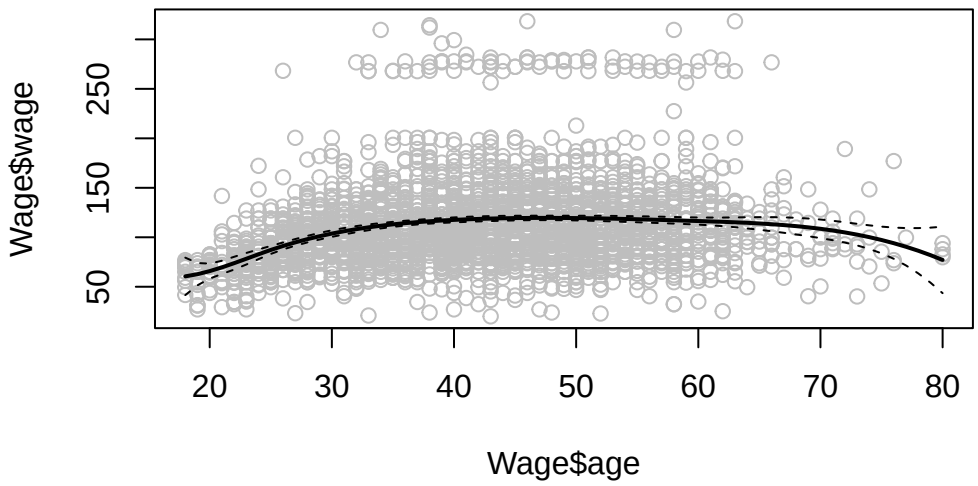
Problem 6: Splines (1.5 pts total)

Part a (Code: 0.5 pts)

Run the first three chunks of code in ISLR Lab 7.8.2 (covering regression splines).

Solution

```r
library(splines)
fit <- lm(Wage$wage ~ bs(age, knots = c(25, 40, 60)), data = Wage)
pred <- predict(fit, newdata = list(age = age.grid), se = T)
plot(Wage$age, Wage$wage, col = "gray")
lines(age.grid, pred$fit, lwd = 2)
lines(age.grid, pred$fit + 2 * pred$se, lty = "dashed")
lines(age.grid, pred$fit - 2 * pred$se, lty = "dashed")
```




```r
dim(bs(Wage$age, knots = c(25, 40, 60)))
```

```
[1] 3000    6
```

```r
dim(bs(Wage$age, df = 6))
```

```
[1] 3000    6
```

```r
attr(bs(Wage$age, df = 6), "knots")
```

```
[1] 33.75 42.00 51.00
```

```r
#fit2 <- lm(Wage$wage ~ ns(age, df = 4), data = Wage)
#pred2 <- predict(fit2, newdata = list(age = age.grid), se = T)
#lines(age.grid, pred2$fit, col = "red", lwd = 2)
# Put in comments because wouldn't render without plot but was not ordered by instructions to
```

Part b (Code: 1 pt)

Fit a model predicting `overall` from `difficulty` using the `associates` dataset and natural splines with three interior knots at 2, 3, and 4.

Produce a figure similar to that in Problem 5c, but with this natural spline model.

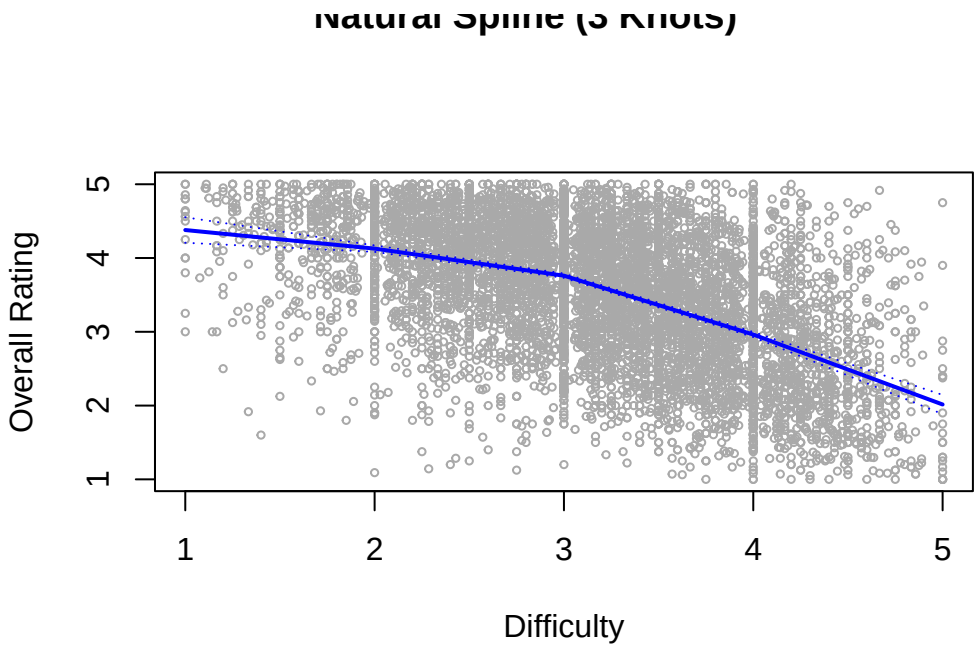
Solution

```
fit_nsoa <- lm(overall ~ ns(difficulty, knots = c(2, 3, 4)),
               data = associates)
coef(summary(fit_nsoa))
```

```
                                     Estimate Std. Error    t value
(Intercept)                          4.3794224 0.08808417  49.718607
ns(difficulty, knots = c(2, 3, 4))1 -0.5147806 0.08228733  -6.255892
ns(difficulty, knots = c(2, 3, 4))2 -1.3346859 0.07322689 -18.226717
ns(difficulty, knots = c(2, 3, 4))3 -2.2558605 0.18966031 -11.894215
ns(difficulty, knots = c(2, 3, 4))4 -2.2242158 0.07510657 -29.614131
                                        Pr(>|t|)
(Intercept)                          0.000000e+00
ns(difficulty, knots = c(2, 3, 4))1  4.154187e-10
ns(difficulty, knots = c(2, 3, 4))2  9.059536e-73
ns(difficulty, knots = c(2, 3, 4))3  2.363833e-32
ns(difficulty, knots = c(2, 3, 4))4 4.828343e-183
```

```
difflims_ns <- range(associates$difficulty)

diff.grid_ns <- seq(from = difflims_ns[1],
                to = difflims_ns[2])

preds_nsoa <- predict(fit_nsoa,
                  newdata = list(difficulty = diff.grid_ns),
                  se = FALSE,
                  interval = "confidence")
```

```
plot(associates$difficulty, associates$overall, xlim = difflims_ns, cex = .5, xlab = "Diffic
title("Natural Spline (3 Knots)", outer = T)
lines(diff.grid_ns, preds_nsoa[, 1], lwd = 2, col = "blue")
matlines(diff.grid_ns, preds_nsoa[, 2:3], lwd = 1, col = "blue", lty = 3)
```



Problem 7: Increasing Complexity (3 pts total)

Part a (Code: 2 pts; Explanation: 1 pt)

The following code creates a *grid* of unique predictor vectors:

```
# expand.grid creates a data frame in which each
# combination of variable values is in a unique row
diff_hot_grid <- expand.grid(
  difficulty = seq(1, 5, by = 0.1),
  hotness = c("cold", "hot")
)
```

Create a model predicting the `overall` rating of associate professors with an interaction term between `hotness` and a nonlinear function of `difficulty`. Adapting the code from either Problem 5c or 6b to include an interaction term will probably be easiest, but you can change the degree of the polynomial or the number/location of knots if you want.

Use the fitted model to make predictions on `diff_hot_grid`. You do not need to include standard errors or confidence intervals; just a vector of predictions is enough for this part.

Create a data frame combining the *predictor* values (from `diff_hot_grid`) and the *predicted* overall ratings, so that you can more easily match up each combination of predictors to the

corresponding predicted overall rating. Using that data frame, produce a figure similar to the one I produced in Problem 4a, with the following differences:

- I will show in my solutions how to adapt the code from Problem 4b to create the plot, but I do not expect that you can figure this out unless you are already very familiar with the `ggplot2` package. Instead, copy your scatterplot code from Problem 5c and do not worry about changing the point colors or adding a legend.
- The lines on the plot should show the predictions of *your* fitted model for "hot" professors (in red) and "cold" professors (in blue), not the model from Problem 4b.

Explain what this graph suggests about how overall rating changes with difficulty for "hot" vs. "cold" professors.

HINT: I have verified that it is possible to solve this problem by adapting only the code I have written in this lab and the code you have copied from the book labs (you may need to figure out what exactly to pass for `newdata`); however, if you can think of a different way to execute a step (e.g., by adapting code you learned in 338 or 439), go for it.

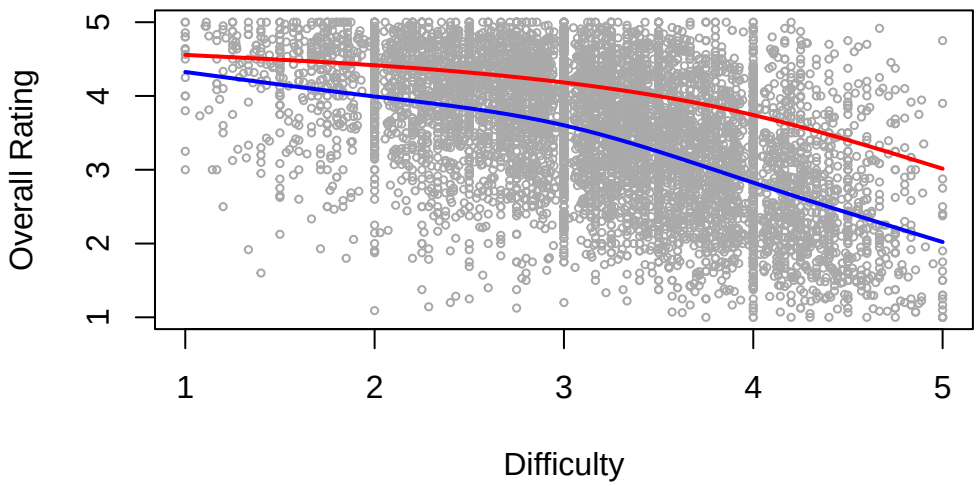
Solution

```
fit_group <- lm(
  overall ~ hotness * ns(difficulty, knots = c(2, 3, 4)),
  data = associates
)
```

```
diff_hot_grid <- expand.grid(
  difficulty = seq(1, 5, by = 0.1),
  hotness = c("cold", "hot")
)

preds_group <- predict(fit_group, newdata = diff_hot_grid)

pred_comb <- data.frame(diff_hot_grid, predicted_overall = preds_group)
```

```
plot(associates$difficulty, associates$overall, xlim = c(1, 5), cex = .5, xlab = "Difficulty"

cold <- subset(pred_comb, hotness == "cold")
lines(cold$difficulty,
      cold$predicted_overall,
      col = "blue",
      lwd = 2)
```

```r
hot <- subset(pred_comb, hotness == "hot")
lines(hot$difficulty,
      hot$predicted_overall,
      col = "red",
      lwd = 2)
```



What this graph suggests about how overall rating changes with difficulty for "hot" vs. "cold" professors is the same old song repeated various times throughout this lab. Attractive professors tend to have higher overall ratings over unattractive professors which also suggests attractive professors are penalized less.